Thème 3 — Analyse de Données en Python, NumPy, Matplotlib et Chaîne de Mesure NTC

Synthèse intégrale des 72 pages du support : Environnement reproductible uv, Notebooks Jupyter, VS Code, gestion des chemins pathlib, calcul vectorisé NumPy, Pandas, tracés scientifiques Matplotlib, étalonnage affine (Exercice 1) et inversion de sonde NTC en pont diviseur (Exercice 2).

> **Structure de ce document de révision :** Ce document couvre 100 % des 72 diapositives du support *Introduction à Python, NumPy et Matplotlib*. Il est structuré en quatre parties : **(1) Résumé complet de la théorie informatique et physique (Python, uv, Jupyter, NumPy, Matplotlib, Thermistance NTC)**, **(2) Formulaire mathématique et guide décisionnel (« Quelle formule utiliser et quand ? »)**, **(3) Résolution détaillée de tous les exercices intermédiaires et des 2 grands exercices de synthèse avec scripts complets**, et **(4) Aide-mémoire exhaustif des fonctions susceptibles de tomber à l'examen**.

PARTIE 1 — Résumé Complet de la Théorie (Python Scientifique & Capteurs)

1.1 Pourquoi Python en Instrumentation ? Comparaison des Outils et Usage de l'IA

En instrumentation industrielle et scientifique, le traitement des données acquises peut s'effectuer avec différents outils logiciels (Slide 3) :

- **Excel :** Très intuitif au premier abord, mais devient vite lourd, source d'erreurs et compliqué dès que les formules mathématiques ou les volumes de données deviennent complexes.
- **LabVIEW :** Programmation graphique (G), excellent support d'interfaces matérielles en tout genre, principalement orienté acquisition temps réel et bancs de test.
- **MATLAB :** Un standard historique très puissant disposant de nombreuses librairies spécialisées, mais propriétaire et très coûteux (« un standard pour les riches ! »).
- **Python :** Langage générique, **gratuit et open-source**, polyvalent, doté d'une immense communauté et de librairies scientifiques de référence (`numpy` , `matplotlib` , `pandas`).

Les 5 compétences cibles du module Python (Slide 4) : (1) Configurer un environnement reproductible avec `uv` ; (2) Importer des fichiers CSV en tableaux `numpy` ou DataFrames `pandas` ; (3) Tracer des signaux avec `matplotlib` selon les règles de l'art (titre, labels, unités, échelles, légende) ; (4) Manipuler des tables de mesures avec `pandas` ; (5) Effectuer des opérations vectorisées avec `numpy` (inversion d'une caractéristique affine ou non linéaire, statistiques descriptives).

Gradation des 4 modes d'utilisation de l'IA (Slide 5) :

1. **No GenAI (Reference Mode) :** Usage de l'IA interdit (consultation uniquement de la documentation officielle). *C'est le mode des tests et examens !*
2. **Tutor Mode :** L'IA explique, définit, pose des questions, donne du feedback et aide à l'apprentissage (recommandé pendant l'étude du cours).
3. **Collab Mode :** L'IA génère des brouillons, des structures ou des extraits de code (toléré en support lors des travaux pratiques).
4. **Agentic Mode :** L'IA réalise les tâches et prend les décisions de façon autonome à la place de l'étudiant (proscrit pour acquérir la maîtrise des scripts).

1.2 Gestion d'Environnement Reproductible avec `uv`, Jupyter et VS Code

Pourquoi utiliser `uv` ? (Slides 7 à 12) : `uv` est un gestionnaire moderne et ultra-rapide de paquets et d'environnements Python qui remplace la combinaison traditionnelle `pip` + `venv` + `virtualenv` + `pyenv`. Il permet d'installer une version précise de Python (ex. `3.12`), crée automatiquement un dossier d'environnement virtuel isolé `.venv/`, déclare les dépendances du projet dans le fichier **`pyproject.toml`** et fige les versions exactes de tous les sous-paquets dans un fichier de verrouillage **`uv.lock`**. Ainsi, n'importe quel collègue ou enseignant récupérant le projet recrée exactement le même environnement au bit près.

- **Exécution avec `uv run` :** La commande `python script.py` n'utilise que le Python global du système (souvent absent ou dépourvu des librairies). Au lieu d'activer manuellement l'environnement virtuel (`.venv\Scripts\Activate.ps1`), la commande **`uv run python mon_script.py`** (ou `uv run mon_script.py`) vérifie automatiquement que l'environnement `.venv` est synchronisé avec `pyproject.toml` avant chaque lancement.
- **Conventions d'importation universelles (Slide 13) :**
  — `import numpy as np` : Calcul numérique vectorisé et tableaux multidimensionnels (`ndarray`).
  — `import matplotlib.pyplot as plt` : Tracé et mise en forme de graphiques scientifiques.
  — `import pandas as pd` : Manipulation de données tabulaires (colonnes nommées, types mixtes, filtrage).
- **Les Notebooks Jupyter (`.ipynb`) (Slides 17 à 23) :**
  — Un notebook combine des **cellules de code Python** (exécutées une par une avec `Shift+Enter`, affichant le résultat ou le graphique juste en dessous) et des **cellules de texte Markdown** (titres `#`, sous-titres `##`, listes, équations LaTeX entre `$$...$$`).
  — *Piège majeur de l'ordre d'exécution (Slide 21) :* Dans un notebook, une variable créée dans une cellule reste stockée dans la mémoire du noyau (*Kernel*) jusqu'à son redémarrage, même si la cellule a été supprimée ou si les cellules ont été exécutées dans le désordre ! Pour garantir qu'un notebook est **reproductible**, il faut placer tous les imports au début et toujours vérifier l'exécution linéaire avec **Kernel > Restart Kernel and Run All Cells**.
  — *Exportation d'un notebook en PDF (Slides 22–23) :* La commande `uv run jupyter nbconvert --to pdf exploration.ipynb` requiert l'installation préalable de **3 outils complémentaires** : (1) le paquet Python **`nbconvert`** (`uv add nbconvert`), (2) le convertisseur universel de documents **`Pandoc`** (`pandoc --version`), et (3) une distribution **LaTeX** telle que **`MiKTeX`** sous Windows (fournissant le compilateur `pdflatex --version`).

- **Développement et débogage dans VS Code (Slides 25 à 29) :**
  — Les fichiers `.py` utilisent l'**Interpréteur Python** sélectionné (`Ctrl+Shift+P` → *Python: Select Interpreter* → `.venv\Scripts\python.exe`), tandis que les notebooks `.ipynb` utilisent le **Kernel Jupyter** (bouton *Select Kernel* en haut à droite → choisir `.venv` après avoir installé `ipykernel`).
  — *Comment lire une erreur (Traceback) dans la console ? (Slide 29) :* Un message d'erreur Python (*Traceback*) se lit toujours **de bas en haut** : la **toute dernière ligne** indique le type exact de l'erreur (ex. `FileNotFoundError` , `NameError` , `ModuleNotFoundError`) et son explication, tandis que les lignes situées juste au-dessus indiquent le fichier `.py` et le numéro de ligne précis où le programme a planté.

1.3 Organisation des Fichiers, Chemins `pathlib` et Importation CSV

Un projet d'instrumentation sépare toujours le code et les mesures dans des sous-dossiers distincts, typiquement `mon_projet/script/analyse.py` et `mon_projet/data/mesures.csv` (Slide 32).

- **Le piège du répertoire de travail (Slides 15, 32, 33) :** Un chemin relatif simple comme `"data/mesures.csv"` dépend du dossier depuis lequel le terminal lance Python (donné par `Path.cwd()`). Si l'on lance le script depuis un autre dossier, Python lève une erreur `FileNotFoundError` .
- **La solution robuste universelle pour les scripts `.py` et les notebooks :** On utilise la bibliothèque standard `pathlib` et la variable spéciale `__file__` qui contient le chemin du script en cours d'exécution. Comme `__file__` n'existe pas dans une console interactive ou dans certains notebooks Jupyter (ce qui lèverait une `NameError`), on utilise la structure de protection `try / except NameError` (Slides 58 et 71) :

```
from pathlib import Path
try:
    script_dir = Path(__file__).resolve().parent  # Dossier exact contenant le
script .py
except NameError:
    script_dir = Path.cwd()                       # Repli pour Jupyter / console
interactive

chemin_csv = script_dir / "../data/etalonnage_capteur.csv"
```

- **Les 3 méthodes pour charger un fichier CSV de mesures (Slides 34 à 36) :**
  1. **`np.loadtxt(chemin, delimiter=",", skiprows=1, unpack=True)` :** Méthode NumPy rapide et directe. Comme NumPy ne manipule que des nombres, `skiprows=1` est **obligatoire** pour ignorer la 1re ligne contenant les titres des colonnes, et `unpack=True` permet d'extraire directement chaque colonne dans une variable séparée (`X_ref, V_etal = np.loadtxt(...)`).
  2. **`np.genfromtxt(chemin, delimiter=",", names=True)` :** Alternative NumPy plus tolérante (gère les valeurs manquantes) qui lit les noms de l'en-tête et permet d'accéder aux colonnes par leur nom : `donnees["X_ref_bar"]` .
  3. **`df = pd.read_csv(chemin)` (Pandas) :** Idéal pour les bases de données tabulaires mêlant texte et nombres. On inspecte les 5 premières lignes avec `df.head()` , la liste des colonnes avec `df.columns.tolist()` , et on reconvertit une colonne en vecteur NumPy avec **`X_ref = df["X_ref_bar"].to_numpy()`** .

1.4 Calcul Vectorisé avec NumPy et Règles de l'Art Graphique avec Matplotlib

- **Différence fondamentale entre `np.arange` et `np.linspace` (Slide 38) :**
  — `np.arange(debut, fin, pas)` : on impose le **pas** d'incrémentation, le nombre de points en découle, et la **borne finale est EXCLUE** (ex. `np.arange(0, 10, 2)` donne `[0, 2, 4, 6, 8]`).
  — `np.linspace(debut, fin, n)` : on impose le **nombre total de points n** régulièrement espacés, le pas en découle, et la **borne finale est INCLUSE** (ex. `np.linspace(0, 10, 5)` donne `[0., 2.5, 5., 7.5, 10.]`). C'est la fonction privilégiée pour créer un axe continu afin de tracer une courbe théorique lisse.
- **Pourquoi utiliser des opérations vectorisées NumPy plutôt qu'une boucle `for` ? (Slide 39) :**
  Écrire `X_estime = (V_mesure - b) / a` applique la transformation affine inverse simultanément à tous les éléments du vecteur `ndarray`. Comparée à une liste en compréhension `[(v - b) / a for v in V_mesure]`, l'opération vectorisée NumPy est : (1) **beaucoup plus courte et lisible** (identique à la formule mathématique), (2) **beaucoup plus rapide** (exécutée en code C compilé et non interprétée élément par élément), et (3) **conserve le type `ndarray`** (alors que la compréhension de liste renvoie une `list` Python sur laquelle on ne peut plus faire de calculs mathématiques directs).
- **Statistiques descriptives et piège des unités (Slides 41–42) :**
  Les fonctions `np.min(v)`, `np.max(v)`, `np.mean(v)` (moyenne arithmétique) et `np.std(v)` (écart-type = √variance) s'expriment toutes dans la **même unité que les données** (ex. en Volts `[V]`). En revanche, la variance **`np.var(v)`** (dispersion quadratique moyenne autour de la moyenne) s'exprime dans l'**unité des données AU CARRÉ** (ex. en `[V²]`) !
- **Règles de l'art d'un graphique Matplotlib pour l'examen et les TPs (Slides 44 à 52) :**
  1. *Formalisme obligatoire :* Tout graphique scientifique doit comporter un **titre explicite** (`ax.set_title`), des **labels sur les deux axes précisant la grandeur et son unité entre crochets** (ex. `ax.set_xlabel("Pression [bar]")`, `ax.set_ylabel("Tension [V]")`), une **grille** discrète (`ax.grid(True, alpha=0.3)`) et une **légende** (`ax.legend(loc="best")`).
  2. *Points discrets vs Courbe de modèle (Slide 52) :* Lorsqu'on dispose d'un petit nombre de points de mesure (ex. les 11 points d'étalonnage), il ne faut **jamais les relier par des segments avec `ax.plot`** (ce qui créerait une ligne brisée trompeuse). On affiche les mesures expérimentales sous forme de **nuage de points non reliés avec `ax.scatter(X_ref, V_etal)`** (éventuellement accompagnés de leurs barres d'incertitude `ax.errorbar`), et on superpose la caractéristique théorique ou estimée sous forme de **ligne continue avec `ax.plot`** calculée sur un vecteur dense `np.linspace`.
  3. *Sauvegarde en PNG (Slide 46) :* Toujours appeler `plt.tight_layout()` puis `fig.savefig("chemin.png", dpi=150, bbox_inches="tight")` **AVANT** `plt.show()`, car `plt.show()` vide la figure de la mémoire une fois la fenêtre fermée (sauvegarder après produirait une image blanche !).

1.5 Théorie de la Sonde Thermistance NTC et de sa Chaîne de Conditionnement (Slides 60 à 64)

Contrairement aux sondes métalliques (comme la Pt100 dont la résistance augmente de façon quasi linéaire avec la température), une **thermistance NTC** (*Negative Temperature Coefficient*) voit sa résistance **diminuer fortement et exponentiellement lorsque la température augmente**. Elle est modélisée par la **loi Beta** :

> **Modèle physique de la thermistance NTC (Loi Beta) :**
> $\boldsymbol{R(T) = R_0 \cdot \exp[\, B \cdot (1/T - 1/T_0)\,]}$
> • T et T_0 : Températures absolues exprimées **obligatoirement en Kelvins [K]** (T [K] = T [°C] + 273,15).
> • R_0 : Résistance nominale à la température de référence T_0 (dans le cours : R_0 = 10 000 Ω à T_0 = 25 °C = 298,15 K).
> • B : Coefficient caractéristique de la sonde en [K] (typiquement entre 3000 et 4500 K ; dans le cours : B = 3950 K).

Conditionnement en pont diviseur de tension (Slide 62) : Comme un système d'acquisition (ADC) ne sait mesurer que des tensions et non directement des résistances, on place la sonde NTC dans un pont diviseur de tension alimenté par V_{cc} = 5 V avec une résistance connue R_{fixe} = 10 000 Ω choisie du **même ordre de grandeur que R_0 afin d'obtenir la sensibilité maximale autour du point de fonctionnement T_0**.

> **Chaîne complète de mesure et de traitement inverse (Figure 1, Slide 63) :**
> T_{eau} (grandeur physique réelle inconnue) → *[Pont diviseur]* → V_{out} (**seule grandeur mesurée**, enregistrée dans la colonne `V_pont_V` du CSV) → *[Étape 1 : Inversion du pont]* → R_{th} (calculée par le script) → *[Étape 2 : Inversion de la loi Beta]* → T_{th} (calculée par le script) → *[Export]* → **Résultat graphique**.

PARTIE 2 — Formulaire Complet et Guide Décisionnel (« Quelle formule utiliser et quand ? »)

Modèle / Étape	Formule Mathématique & Traduction NumPy	Quand l'utiliser à l'examen ? & Pièges à éviter
Caractéristique affine directe d'un capteur	$V_{mesure} = a \cdot X + b$ NumPy : `V = a * X + b`	**Quand :** Modéliser la tension de sortie d'un capteur linéaire de gain (sensibilité) a [V/bar] et d'offset b [V].

Modèle / Étape	Formule Mathématique & Traduction NumPy	Quand l'utiliser à l'examen ? & Pièges à éviter
Identification de $\hat{a}$ et $\hat{b}$ (Étalonnage)	• *2 points extrêmes (X_0, V_0) et (X_n, V_n) :* $\hat{a} = (V_n - V_0) / (X_n - X_0)$; $\hat{b} = V_0 - \hat{a} \cdot X_0$ • *Régression moindres carrés (tous les points) :* `a_hat, b_hat = np.polyfit(X_ref, V_etal, 1)`	**Quand :** Retrouver les vrais paramètres d'un capteur ayant dérivé à partir d'un essai d'étalonnage. **Avantage moindres carrés :** Insensible au bruit d'un point isolé car il moyenne les erreurs sur les N points.
Correction affine inverse (Mesure → Grandeur)	$\hat{X}(t) = (V_{mesure}(t) - \hat{b}) / \hat{a}$ NumPy : `X_estime = (V_mesure - b_hat) / a_hat`	**Quand :** Convertir un signal temporel de tension brute `V_mesure` en grandeur physique corrigée `X_estime` (en bar).
Pont diviseur direct (Sonde NTC au GND)	$V_{out} = V_{cc} \cdot R_{ntc} / (R_{fixe} + R_{ntc})$	**Quand :** Calculer la tension délivrée par le pont connaissant R_{ntc}. **Remarque :** Comme R_{ntc} est au numérateur (et croît plus vite que le dénominateur), quand T augmente → R_{ntc} diminue → V_{out} **diminue**.
Étape 1 — Inversion du pont diviseur ($V_{out} \rightarrow R_{ntc}$)	$V_{out} \cdot (R_{fixe} + R_{ntc}) = V_{cc} \cdot R_{ntc}$ $\Rightarrow$ $R_{ntc} = R_{fixe} \cdot V_{out} / (V_{cc} - V_{out})$ NumPy : `R = Rfixe * Vout / (Vcc - Vout)`	**Quand :** Première étape obligatoire pour remonter de la tension mesurée V_{out} à la résistance de la thermistance R_{ntc}. **Piège :** Attention au signe au dénominateur : c'est bien ($V_{cc} - V_{out}$).
Étape 2 — Inversion de la loi Beta ($R_{ntc} \rightarrow T$)	$1/T = 1/T_0 + (1/B) \cdot \ln(R_{ntc} / R_0)$ $\Rightarrow$ T **[K] = [1/T_0 + (1/B) · ln(R_{ntc} / R_0)] −1** $\Rightarrow$ T **[°C] =** T **[K] − 273,15** NumPy : `1.0 / (1.0/T0_K + np.log(R/R0)/B) - 273.15`	**Quand :** Seconde étape pour convertir R_{ntc} en température. **Piège mortel :** T_0 doit impérativement être en **Kelvins** ($T_{0,K}$ = 25 + 273,15 = 298,15 K) dans la formule, et en Python le logarithme népérien ln(x) s'écrit **`np.log(x)`** (et non `np.ln` qui n'existe pas !).

PARTIE 3 — Résolution Détaillée de Tous les Exercices et Scripts Corrigés

3.1 Résolution des 5 Exercices Pratiques Intermédiaires (Chapitres 2.2 à 2.5)

1. **Exercice 2.2.7 (Slide 14 — L'austère ligne de commande) :**
   La commande `uv run --with matplotlib --with numpy python` démarre une session Python interactive en chargeant à la volée les bibliothèques `matplotlib` et `numpy` dans un environnement éphémère isolé. Lorsque l'on crée `tableau = np.array([1, 2, 3, 4, 5])` et que l'on tape `tableau * 2`, NumPy effectue une **multiplication vectorisée élément par élément** et retourne `array([2, 4, 6, 8, 10])`. Ensuite, `plt.plot(tableau, marker="o")` trace les 5 valeurs en fonction de leur indice en marquant chaque point d'un cercle.

2. `[0, 1, 2, 3, 4]`
3. **Exercice 2.2.8 (Slide 15 — La sordide histoire du répertoire de travail) :**
   Le script utilise `Path.cwd()` (*Current Working Directory*) qui affiche le dossier depuis lequel l'utilisateur a exécuté la commande dans son terminal (et non le dossier où est enregistré le fichier `.py`). Si on lance `uv run python script/mon_code.py` depuis la racine `mon_projet/`, `Path.cwd()` renvoie `.../mon_projet` ; mais si on fait `cd script` avant de lancer `uv run python mon_code.py`, `Path.cwd()` renvoie `.../mon_projet/script`. C'est pourquoi il ne faut jamais se fier uniquement à `Path.cwd()` pour ouvrir un fichier de données relatif.
4. **Exercice 2.4.1 (Slide 26 — Premier pas avec VS Code : script `test.py`) :**
   Énoncé : Créer un script `test.py` qui importe `numpy`, génère un vecteur NumPy avec les nombres de 1 à 10, calcule et affiche la somme (fonction `np.sum`).

```
import numpy as np

# Génération du vecteur d'entiers de 1 à 10 inclus (borne 11 exclue avec arange)
vecteur = np.arange(1, 11)   # Ou np.linspace(1, 10, 10)
somme = np.sum(vecteur)
print("Vecteur :", vecteur)  # [ 1  2  3  4  5  6  7  8  9 10]
print("Somme   :", somme)    # 55
```

5. **Exercice 2.5.3 (Slide 34 — Lecture automatique de `etalonnage_capteur.csv` depuis `/script`) :**
   Énoncé : Écrire un script sauvé dans `/script` qui trouve par lui-même le chemin vers `../data/etalonnage_capteur.csv` en se basant sur `Path(__file__).resolve().parent` :

```
import numpy as np
from pathlib import Path

script_dir = Path(__file__).resolve().parent
chemin_csv = script_dir / "../data/etalonnage_capteur.csv"

donnees = np.loadtxt(chemin_csv, delimiter=",", skiprows=1)
print("Format (lignes, colonnes) :", donnees.shape)  # (11, 2)
print("Données :\n", donnees)
```

6. **Exercice 2.5.9 (Slide 40 — Break pratique : Sinus d'angles entre 0 et 2π) :**
   Énoncé : Créer un tableau NumPy qui représente des angles entre 0 et 2π, puis utiliser `np.sin` pour calculer le sinus de ces angles :

7.
```
import numpy as np
import matplotlib.pyplot as plt

angles = np.linspace(0, 2 * np.pi, 100)  # 100 angles régulièrement espacés de 0
à 2*pi inclus
y_sin = np.sin(angles)                   # Calcul vectorisé du sinus sur tout le
tableau

plt.plot(angles, y_sin, color="tab:blue", label="sin(x)")
plt.xlabel("Angle [rad]")
plt.ylabel("Amplitude [-]")
plt.title("Fonction sinus sur [0, 2π]")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

3.2 Exercice 1 (Section 2.7, Slides 54 à 58) : Correction d'une Caractéristique Affine (Capteur de Pression 0–10 bar)

Contexte et données de l'énoncé : Un capteur de pression (0–10 bar) délivre une tension liée à la pression X par une relation affine $V_{\text{mesure}} = a \cdot X + b$. Le datasheet constructeur annonce une sensibilité $a_{\text{datasheet}}$ = 0,500 V/bar et un offset $b_{\text{datasheet}}$ = 0,000 V, mais le capteur réel a dérivé. On dispose de :

— `data/etalonnage_capteur.csv` (colonnes `X_ref_bar`, `V_mesure_V` : 11 points pour X_{ref} = 0, 1, …, 10 bar ; d'après les slides 36 et 41 : à 0 bar, V = 0,3144 V ; à 1 bar, V = 0,7407 V ; à 2 bar, V = 1,1920 V ; à 3 bar, V = 1,6305 V ; à 4 bar, V = 2,0444 V ; et à 10 bar, V_{max} = 4,689 V).

— `data/acquisition_capteur.csv` (colonnes `t_s`, `V_mesure_V` : signal temporel acquis lors d'une montée puis descente de pression).

Démarche de résolution détaillée (Questions 1 à 6) :

1. **Identification des paramètres $\hat{a}$ (gain) et $\hat{b}$ (offset) de la droite d'étalonnage (Question 2) :**
   On peut déterminer $\hat{a}$ et $\hat{b}$ de deux manières :

   > **• Méthode A — Par les 2 points extrêmes (X_0 = 0 bar, V_0 = 0,3144 V) et (X_{10} = 10 bar, V_{10} = 4,689 V) :**
   > $\hat{b} = V_0 =$ **0,3144 [V] ≈ 0,314 [V]**
   > $\hat{a} = (V_{10} - V_0) / (X_{10} - X_0) = (4{,}689 - 0{,}3144) / (10{,}0 - 0{,}0) = 4{,}3746 / 10 \approx$ **0,4375 [V/bar] ≈ 0,437 [V/bar]**.
   >
   > **• Méthode B — Par régression linéaire aux moindres carrés sur les 11 points (méthode optimale) :**
   > En Python, `a_hat, b_hat = np.polyfit(X_ref, V_etal, 1)` ajuste la droite minimisant la somme des carrés des écarts sur les 11 points et donne **$\hat{a}$ ≈ 0,437 [V/bar]** et **$\hat{b}$ ≈ 0,314 [V]** (valeurs confirmées à la slide 39 du cours !).

2. **Déduction de la relation de correction inverse (Questions 3 & 4) :**
   En isolant X dans $V_{\text{mesure}} = \hat{a} \cdot X + \hat{b}$, on obtient la formule permettant de calculer la pression réelle estimée $\hat{X}(t)$ à partir du vecteur temporel de tension brute `V_acq` :

   > $$\hat{X}(t) = (V_{\text{mesure}}(t) - \hat{b}) / \hat{a} = (V_{\text{mesure}}(t) - 0{,}314) / 0{,}437 \text{ [bar]}$$

3. **Comparaison au datasheet et critique de la méthode des 2 points extrêmes (Question 6) :**
   — *Comparaison :* Le gain réel est plus faible que celui annoncé ($\hat{a}$ = 0,437 V/bar contre $a_{\text{datasheet}}$ = 0,500 V/bar, soit une perte de sensibilité de −12,6 %) et il existe un offset non nul ($\hat{b}$ = +0,314 V contre 0,000 V). Si l'on utilisait naïvement la formule du datasheet ($X = V$ / 0,5), à X = 0 bar on lirait 0,314 / 0,5 = **0,63 bar d'erreur**, et à 10 bar on lirait 4,689 / 0,5 = **9,38 bar** (erreur de −0,62 bar) !
   — *Origine physique de l'écart :* Tolérances de fabrication du capteur, vieillissement mécanique de la membrane sensible, précontrainte résiduelle au montage mécanique (qui crée l'offset b), et dérive thermique de l'électronique de conditionnement.
   — *Pourquoi peut-on faire beaucoup mieux que la méthode des 2 points extrêmes ?* La méthode des 2 points extrêmes ignore totalement les 9 points de mesure intermédiaires (X = 1 à 9 bar). **Si l'un des deux points extrêmes (X = 0 ou X = 10 bar) est entaché d'une erreur de mesure inhabituellement grande (bruit fort ou valeur aberrante), la pente $\hat{a}$ et l'offset $\hat{b}$ sont complètement faussés sur toute l'étendue de mesure !** En utilisant la **régression linéaire par moindres carrés (`np.polyfit(X_ref, V_etal, 1)`)**, on exploite l'information des 11 points simultanément, ce qui moyenne le bruit aléatoire de mesure et réduit l'incertitude sur $\hat{a}$ et $\hat{b}$ d'un facteur proportionnel à $\sqrt{N}$.

Script Python complet corrigé pour l'Exercice 1 :

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

try:
    script_dir = Path(__file__).resolve().parent
except NameError:
    script_dir = Path.cwd()

# 1. Chargement des données d'étalonnage
X_ref, V_etal = np.loadtxt(
    script_dir / "../data/etalonnage_capteur.csv", delimiter=",", skiprows=1,
unpack=True
)

# 2. Identification de a et b : comparaison 2 points extrêmes vs Moindres carrés
a_extremes = (V_etal[-1] - V_etal[0]) / (X_ref[-1] - X_ref[0])
b_extremes = V_etal[0] - a_extremes * X_ref[0]

a_hat, b_hat = np.polyfit(X_ref, V_etal, 1)  # Régression linéaire optimale sur les
11 points
print(f"Méthode 2 points extrêmes : a = {a_extremes:.4f} V/bar, b =
{b_extremes:.4f} V")
print(f"Moindres carrés (polyfit) : a = {a_hat:.4f} V/bar, b = {b_hat:.4f} V")

# Tracé de la courbe d'étalonnage (Figure 1)
X_continu = np.linspace(np.min(X_ref), np.max(X_ref), 100)
V_estime_continu = a_hat * X_continu + b_hat
V_datasheet_continu = 0.500 * X_continu + 0.000

fig1, ax1 = plt.subplots(figsize=(8, 4.5))
ax1.scatter(X_ref, V_etal, color="tab:blue", s=40, zorder=3, label="Points
mesurés")
ax1.plot(X_continu, V_estime_continu, color="tab:orange", lw=1.5,
label="Caractéristique estimée")
ax1.plot(X_continu, V_datasheet_continu, "--", color="gray", lw=1.2,
label="Caractéristique datasheet")
ax1.set_title("Étalonnage du capteur de pression (0–10 bar)")
ax1.set_xlabel("Pression [bar]")
ax1.set_ylabel("Tension [V]")
ax1.grid(True, alpha=0.3)
ax1.legend(loc="best")
fig1.tight_layout()

# 3 & 4. Chargement du signal temporel et application de la correction inverse
vectorisée
t_s, V_mesure = np.loadtxt(
    script_dir / "../data/acquisition_capteur.csv", delimiter=",", skiprows=1,
unpack=True
)
X_corrige = (V_mesure - b_hat) / a_hat  # Inversion vectorisée en bar

# 5. Tracé sur une figure à 2 sous-graphiques partageant l'axe temporel
```

3.3 Exercice 2 (Section 2.9, Slides 66 à 72) : Sonde NTC dans une Bouilloire

Contexte et paramètres de l'énoncé : On suit la température de l'eau dans une bouilloire avec une sonde NTC (R_0 = 10 000 Ω à T_0 = 25 °C = 298,15 K, B = 3950 K) montée dans un pont diviseur (V_{cc} = 5,0 V, R_{fixe} = 10 000 Ω). Le fichier `data/sonde_ntc_bouilloire.csv` contient ~600 points relevés toutes les 5 s pendant 50 minutes (colonnes `t_s`, `V_pont_V`), avec un bruit électronique d'écart-type 10 mV. L'eau, initialement à 10 °C, chauffe jusqu'à ce que le thermostat coupe l'alimentation à 90 °C, puis refroidit lentement vers l'ambiante.

Démarche de résolution et calculs détaillés (Questions 1 à 6) :

1. **Vérification analytique des points clés de la courbe brute $V_{out}(t)$ (Slide 69) :**
   Sur la Figure 2 du cours (Slide 69), on observe que V_{out} part de **~3,34 V** à t = 0 min, descend jusqu'à un minimum de **~0,435 V** vers $t \approx 9{,}2$ min, puis remonte progressivement jusqu'à **~2,28 V** à t = 50 min. Vérifions par le calcul à quelles températures correspondent ces tensions :

   > **• À t = 0 min ($V_{out} \approx 3{,}343$ V) :**
   > Étape 1 : $R_{ntc} = 10\,000 \times 3{,}343 / (5{,}0 - 3{,}343) \approx$ **20 175 [Ω]**.
   > Étape 2 : T [K] = [1/298,15 + ln(20 175 / 10 000)/3950]$^{-1}$ = 283,15 K ⇒ $T(0)$ = 283,15 − 273,15 = **10,0 °C** (eau froide initiale !).
   >
   > **• Au minimum de tension ($t \approx 550$–560 s ≈ 9,2 min, $V_{out,min} \approx 0{,}435$ V) :**
   > Étape 1 : $R_{ntc,min} = 10\,000 \times 0{,}435 / (5{,}0 - 0{,}435) \approx$ **952,9 [Ω]**.
   > Étape 2 : T_{max} [K] = [1/298,15 + ln(952,9 / 10 000)/3950]$^{-1}$ = 363,15 K ⇒ T_{max} = **363,15 − 273,15 = 90,0 °C** !
   >
   > **• À t = 50 min ($V_{out} \approx 2{,}28$ V) :**
   > Étape 1 : $R_{ntc} = 10\,000 \times 2{,}28 / (5{,}0 - 2{,}28) \approx$ **8382 [Ω]** ⇒ Étape 2 : T**(50 min) ≈ 29,0 °C** (refroidissement vers la température ambiante de la pièce $T_{amb} \approx 20$–22 °C).

2. **Identification des 3 grandeurs demandées à la Question 5 :**
   - **Instant où le thermostat coupe le chauffage : $t_{coupure}$ ≈ 550 à 560 s (soit ≈ 9,2 à 9,3 minutes)**, déterminé exactement dans le script via `idx_max = np.argmax(T_C)`.
   - **Valeur du maximum de température : T_{max} ≈ 90 °C** (à ±1 °C près selon le bruit de 10 mV).
   - **Constante de temps approximative τ du refroidissement :** Après la coupure (t > 9,2 min), la température décroît exponentiellement de $T_{max} \approx 90$ °C vers la température ambiante asymptotique $T_{amb} \approx 20$ °C selon $T(t) = T_{amb} + (T_{max} - T_{amb}) \cdot e^{-(t - t_{coupure})/\tau}$. Au bout d'une constante de temps ($t = t_{coupure} + \tau$), la température a chuté de **63,2 %** de l'écart total (90 − 20 = 70 °C), et passe donc par la valeur :
     $T(t_{coupure} + \tau) = 20 + 0{,}368 \times (90 - 20) \approx$ **45,8 °C** (ce qui correspond à $V_{out} \approx 1{,}48$ V sur la Figure 2, atteint à $t \approx 30$ min). La constante de temps du refroidissement de la bouilloire vaut donc **τ ≈ 30 − 9,2 ≈ 20 à 21 minutes (≈ 1200 s)**.

3. **Réponse complète à la Question d'analyse 6 : « Pourquoi la relation entre V_{out} et T ne peut-elle pas être approximée par une simple droite sur toute la plage 10–90 °C ? »**
   La relation globale $V_{out}(T)$ cumule **deux non-linéarités sévères** sur une plage aussi large que 80 °C :
   1. *Non-linéarité exponentielle de la NTC (loi Beta) :* La résistance varie d'un facteur supérieur à 21 entre 10 °C (~20 175 Ω) et 90 °C (~953 Ω). Sa sensibilité $dR/dT = -(B/T^2)\cdot R(T)$ passe de −994 Ω/K à 10 °C à seulement −28,5 Ω/K à 90 °C (soit 35 fois plus faible !).
   2. *Non-linéarité homographique du pont diviseur :* La fonction $V_{out} = V_{cc}\cdot R_{ntc}/(R_{fixe} + R_{ntc})$ est quasi linéaire uniquement au voisinage immédiat de son point d'inflexion ($R_{ntc} \approx R_{fixe}$ = 10 kΩ, soit autour de T_0 = 25 °C ± 15 °C). À haute température (70–90 °C), $R_{ntc} \ll R_{fixe}$ et la courbe $V_{out}(T)$ s'écrase asymptotiquement vers 0 V : une droite commettrait des erreurs de plus de 10 à 15 °C, et la faible sensibilité à 90 °C (~15 mV/°C contre ~55 mV/°C à 25 °C) explique d'ailleurs pourquoi le bruit de 10 mV devient beaucoup plus visible au sommet de la courbe de température $T(t)$!

Script Python complet corrigé pour l'Exercice 2 :

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

try:
    script_dir = Path(__file__).resolve().parent
except NameError:
    script_dir = Path.cwd()

# --- Paramètres du montage et de la sonde NTC (donnés dans l'énoncé) ---
Vcc = 5.0           # Tension d'alimentation du pont [V]
R_fixe = 10_000.0   # Résistance fixe du pont [Ohm]
R0 = 10_000.0       # Résistance de la NTC à T0 = 25 °C [Ohm]
T0_C = 25.0         # Température de référence [°C]
T0_K = T0_C + 273.15 # Température de référence en Kelvins [K] (298.15 K)
B = 3950.0          # Coefficient Beta de la NTC [K]

# 2. Étape 1 : Inversion du pont diviseur (V_out -> R_ntc)
def tension_vers_resistance(Vout, Vcc, Rfixe):
    return Rfixe * Vout / (Vcc - Vout)

# 3. Étape 2 : Inversion de la loi Beta (R_ntc -> Température en °C)
def resistance_vers_temperature(R, R0, T0_K, B):
    inv_T_K = (1.0 / T0_K) + (1.0 / B) * np.log(R / R0)
    T_K = 1.0 / inv_T_K
    return T_K - 273.15

# 1 & 4. Chargement du CSV et calcul vectorisé de R(t) puis T(t)
t_s, V_out = np.loadtxt(
    script_dir / "../data/sonde_ntc_bouilloire.csv", delimiter=",", skiprows=1,
unpack=True
)
t_min = t_s / 60.0  # Conversion du temps en minutes pour l'axe des abscisses

R_ntc = tension_vers_resistance(V_out, Vcc, R_fixe)
T_C = resistance_vers_temperature(R_ntc, R0, T0_K, B)

# 5. Identification automatique de la coupure du thermostat et de la constante de
temps
idx_max = np.argmax(T_C)
t_coupure_s = t_s[idx_max]
t_coupure_min = t_min[idx_max]
T_max = T_C[idx_max]

# Estimation de la constante de temps de refroidissement tau (chute de 63.2 % vers
T_amb = 20 °C)
T_amb = 20.0
T_seuil_tau = T_amb + (T_max - T_amb) * np.exp(-1.0)
idx_apres_coupure = np.where((t_min > t_coupure_min) & (T_C <= T_seuil_tau))[0]
tau_min = t_min[idx_apres_coupure[0]] - t_coupure_min if len(idx_apres_coupure) > 0
else np.nan

print(f"Instant de coupure du thermostat : t = {t_coupure_s: 0f} s
```

PARTIE 4 — Aide-Mémoire Exhaustif des Fonctions et Commandes pour l'Examen

Catégorie	Syntaxe Exacte de la Commande / Fonction	Rôle et Points de Vigilance à l'Examen

Gestionnaire `uv` & Jupyter	`uv init nom-projet`
	`uv python install 3.12` & `uv python pin 3.12`
	`uv add numpy matplotlib pandas`
	`uv add --dev jupyter nbconvert`
	`uv run python script.py`
	`uv run jupyter lab`
	`uv run jupyter nbconvert --to pdf note.ipynb`
		• Crée `pyproject.toml`, `uv.lock` et `.venv/`.
		• Exporter un notebook en PDF nécessite : `nbconvert` + `Pandoc` + `MiKTeX (pdflatex)`.
		• Dans Jupyter : *Restart Kernel and Run All Cells* garantit la reproductibilité.

Chemins & Importation de fichiers CSV	`from pathlib import Path`
	`Path.cwd()`
	`script_dir = Path(__file__).resolve().parent`
	`X, Y = np.loadtxt(f, delimiter=",", skiprows=1, unpack=True)`
	`d = np.genfromtxt(f, delimiter=",", names=True)`
	`df = pd.read_csv(f)` ; `df.head()`
	`vec = df["nom_colonne"].to_numpy()`
		• `__file__` n'existe que dans un fichier `.py` (utiliser `try/except NameError` pour Jupyter).
		• `skiprows=1` est indispensable avec `np.loadtxt` pour sauter les titres des colonnes.

Création & Calculs Vectorisés `numpy`	`np.array([1.0, 2.0, 3.0])`
	`np.zeros(n)`, `np.ones(n)`, `np.full(n, val)`
	`np.arange(debut, fin, pas)` (fin exclue)
	`np.linspace(debut, fin, n)` (fin incluse)
	`np.exp(x)`, `np.log(x)`, `np.sin(x)`, `np.sqrt(x)`
	`np.min(x)`, `np.max(x)`, `np.argmax(x)`, `np.sum(x)`
	`np.mean(x)`, `np.var(x)`, `np.std(x)`
	`a, b = np.polyfit(X, Y, 1)`
		• `np.log(x)` calcule le logarithme naturel $\ln(x)$.
		• `np.var(x)` est en **unité au carré** (ex. V^2) ; `np.std(x)` est dans la **même unité** que `x` (ex. V).
		• `np.argmax(x)` renvoie l'**indice** du maximum.

Catégorie	Syntaxe Exacte de la Commande / Fonction	Rôle et Points de Vigilance à l'Examen

Tracés Scientifiques `matplotlib`	`fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10,6), sharex=True)`
	`ax.plot(x, y, color="tab:blue", lw=1, label="...")`
	`ax.scatter(x, y, s=40, alpha=0.8, zorder=3)`
	`ax.errorbar(x, y, yerr=s, fmt="none", capsize=3)`
	`ax.set_title("...")`, `ax.set_xlabel("... [unité]")`
	`ax.set_xlim(a, b)`, `ax.set_ylim(c, d)`
	`ax.xaxis.set_major_locator(MultipleLocator(2))`
	`ax.xaxis.set_minor_locator(AutoMinorLocator(4))`
	`ax.set_xticks([...])`, `ax.set_xticklabels([...])`
	`ax.text(0.98, 0.02, "Nom", transform=ax.transAxes)`
	`ax.grid(which="major", alpha=0.4)`, `ax.legend(loc="best")`
	`fig.tight_layout()`
	`fig.savefig("out.png", dpi=150, bbox_inches="tight")`
	`plt.show()`
		• Utiliser `ax.scatter` pour des points de mesure peu nombreux et `ax.plot` (avec `np.linspace`) pour le modèle théorique.
		• Toujours appeler `fig.savefig(...)` **avant** `plt.show()`.
		• `MultipleLocator(pas)` fixe l'écart entre deux graduations principales ; `AutoMinorLocator(n)` divise chaque intervalle principal en `n` sous-divisions.